Cloud Computing

Foundations and Distributed Systems Theory

Lecture 01 - Part 2

Academic Year 2025/2026

DATA SCIENCE AND ARTIFICIAL INTELLIGENCE – University of Trieste

Recap: Where We Are

Part 1 covered:

- Historical evolution → Cloud is a synthesis of prior paradigms
- Distributed systems fundamentals → Concurrency, no global clock, independent failures
- Eight fallacies → Design assuming networks fail
- CAP theorem introduced → C, A, P defined

Part 2 covers:

CAP trade-offs → Consistency models → Consensus → Fault tolerance → Why cloud?

CP Systems: Consistency+Partition Tolerance

When correctness matters more than availability

During a partition: Refuse requests rather than return stale data

Some users get errors, but no one gets wrong answers

Examples:

- Banking systems – incorrect balance could cause overdrafts
- Inventory systems – selling items that don't exist
- etcd / ZooKeeper – configuration must be consistent
- Google Spanner – global consistency for critical data

AP Systems: Availability+Partition Tolerance

When availability matters more than immediate consistency

During a partition: Serve requests even with potentially stale data

Everyone gets an answer, but it might be outdated

Examples:

- Shopping carts – better to let users shop than show errors
- Social media likes – slightly wrong count is acceptable
- DNS – stale records better than no resolution
- CDNs – serve cached content, update eventually

How to Choose: CP vs AP

Ask yourself this question:

What is worse: a wrong answer or no answer?

Choose CP when:

- Wrong data causes real harm
- Financial transactions
- Regulatory requirements

Choose AP when:

- Stale data is tolerable
- User experience is critical
- Data can be reconciled later

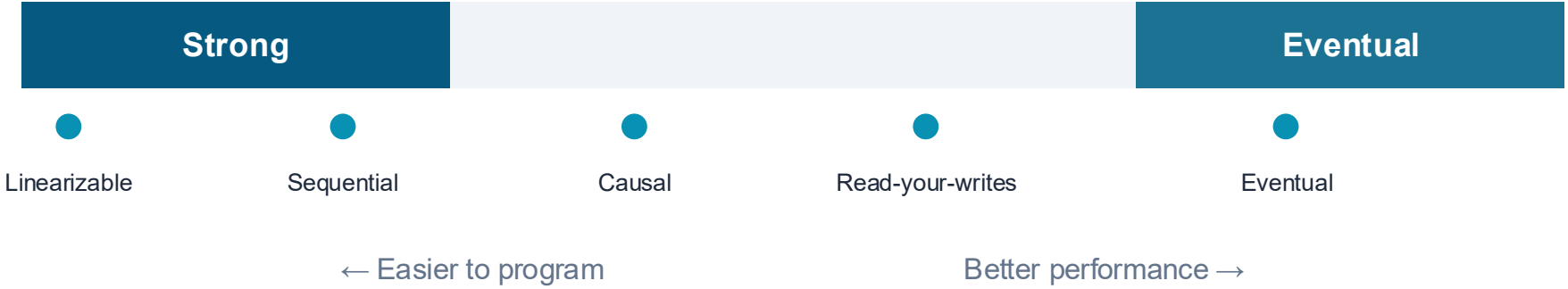
Part 2

Consistency Models

The spectrum between strong and eventual

The Consistency Spectrum

Consistency is not binary – it's a spectrum



Trade-off: Stronger consistency requires more coordination → higher latency

Strong Consistency (Linearizability)

The simplest model to reason about

After a write completes, ALL subsequent reads return that value (or a newer one)

Properties:

- Illusion of single copy of data
- All clients see same order
- Simplest programming model

Costs:

- Requires coordination (consensus)
- Higher latency
- Reduced availability in partitions

Example: Bank account balance must be strongly consistent

Eventual Consistency

The most scalable model

If no new updates are made, eventually all reads will return the same value

Properties:

- Updates propagate asynchronously
- Temporary inconsistency allowed
- System converges over time

Benefits:

- No coordination needed
- Low latency, high throughput
- High availability

Example: Social media like counts – temporary differences are acceptable

The Inconsistency Window

How eventual consistency works in practice

Timeline:

t=0	Write x=5 to Node A	A:5, B:0, C:0
t=1	Read from Node B	Returns 0 (stale)
t=2	A propagates to B	A:5, B:5, C:0
t=3	B propagates to C	A:5, B:5, C:5 ✓

Inconsistency window: t=0 to t=3 (time when reads may return stale data)

Part 3

Consensus and Coordination

How distributed systems agree

The Consensus Problem

Getting multiple nodes to agree on a value

Requirements for consensus:

- Agreement - All correct nodes decide the same value
- Validity - The decided value was proposed by someone
- Termination - All correct nodes eventually decide

Used for:

- Leader election - Who is in charge?
- Distributed transactions – Should we commit?
- Configuration management – What is the current state?

Consensus Algorithms: Paxos and Raft

Paxos (1989/1998)

Leslie Lamport

- First proven consensus algorithm
- Mathematically elegant
- Notoriously difficult to understand
- Hard to implement correctly

Raft (2014)

Ongaro & Ousterhout

- Designed for understandability
- Same guarantees as Paxos
- Easier to implement
- Widely used (etcd, Consul)

Both require a majority (quorum) of nodes to be available

Quorums: The Majority Principle

Why consensus requires a majority

Any two majorities must overlap by at least one node

This overlap ensures information continuity

Nodes (N)	Quorum	Failures Tolerated
3	2	1
5	3	2
7	4	3

Odd numbers preferred: 5 nodes tolerates 2 failures (same as 6 nodes)

Real Example: Kubernetes + etcd

Consensus in production

Kubernetes stores ALL cluster state in etcd

- etcd uses Raft consensus
- Typical production: 5-node etcd cluster
- Can tolerate 2 node failures
- If 3+ nodes fail: Kubernetes control plane unavailable

What happens if etcd loses quorum?

- Cannot schedule new pods
- Cannot update configurations
- Existing workloads continue running (but no changes)

Part 4

Fault Tolerance

Designing for failure

Failure is Normal

In distributed systems, failure is not exceptional.
It is the normal operating condition.

At cloud scale:

- Thousands of servers → Some always failing
- Millions of disks → Disk failures every hour
- Network links → Partitions happen
- Software bugs → Crashes occur

If a node fails, the whole system still work each node play a partial role (partial inputs and outputs)check node status

Redundancy Strategies

Multiple copies protect against failure

Active-Active

All replicas serve requests

✓ No failover delay

✗ Complex consistency

Active-Passive

Standby takes over on failure

✓ Simpler consistency

✗ Failover delay

Quorum-based

Majority must agree

✓ Balanced trade-off

✗ Requires odd nodes

Cloud Provider Redundancy Levels

Availability Zone (AZ) Isolated data center	Tolerates: Single DC failure
Region Multiple AZs in geographic area	Tolerates: AZ failure
Multi-Region Deployment across regions	Tolerates: Regional disaster

Best practice: Deploy across multiple AZs for production workloads

Part 5

Example of Distributed System

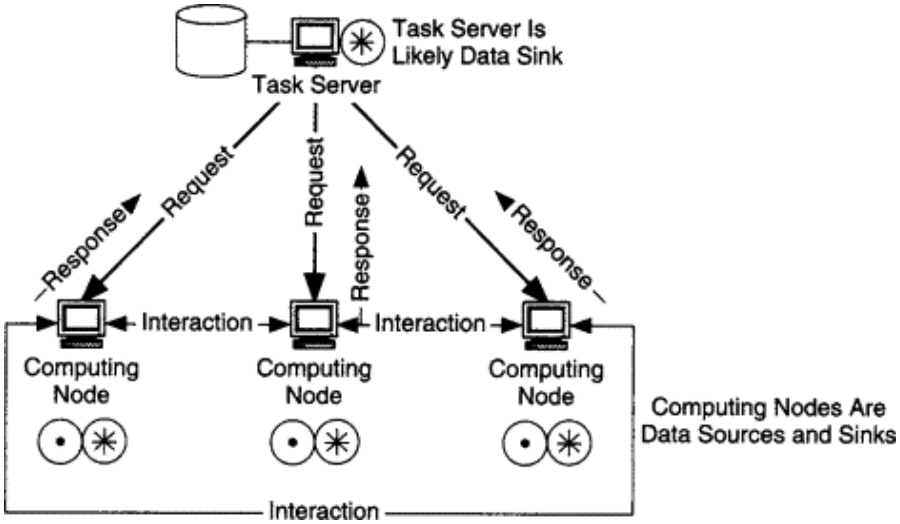
Distributed computing Architectures

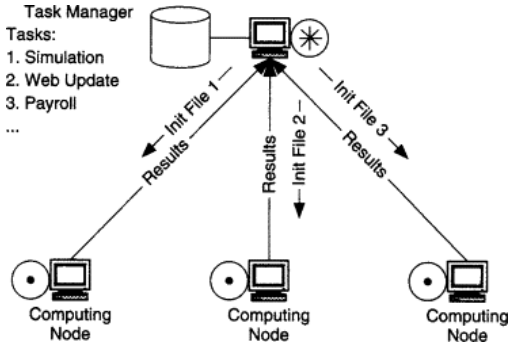
Distributed Architectures

Client Server	→	**Resource management centralized at a server**
3-Tier architecture	→	**move the client intelligence to a middle tier to simplify application development.**
Peer-to-Peer	→	**responsibilities are uniformly divided among all machines, known as peers that serves both as client and servers**

Distributed Architectures

Client Server → **Resource management centralized at a server**



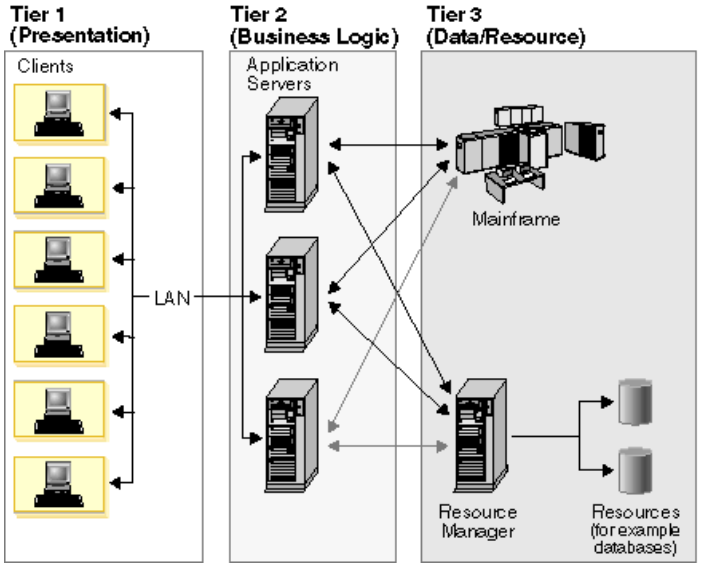


Distributed ARchitectures

3-Tier architecture → **move the client intelligence to a middle tier to simplify application development.**

↓

N-Tier architecture



Part 6

Why Cloud Computing?

The problems it solves

Problems Cloud Computing Solves

CapEx	→	**OpEx**	No upfront investment, pay for usage
Fixed capacity	→	**Elastic scaling**	Scale in minutes, not months
Ops burden	→	**Managed services**	Provider handles infrastructure
Local only	→	**Global reach**	Data centers worldwide

Cloud Trade-offs

Cloud is not a silver bullet

- **Less control:** You don't control hardware, network, or underlying software
- **Vendor lock-in:** Migration between providers is costly and complex
- **Network dependency:** Requires reliable internet; adds latency
- **Cost at scale:** May be more expensive than owned infra for stable workloads
- **Data sovereignty:** Data location subject to provider's data centers

Always evaluate: Is cloud the right choice for THIS workload?

Key Takeaways

1. **CAP theorem constrains design**
   During partitions: choose Consistency OR Availability

2. **Consistency is a spectrum**
   Strong → Eventual, each with trade-offs

3. **Consensus enables coordination**
   Requires quorum; Raft/Paxos in production systems

4. **Failure is normal – design for it**
   Redundancy, multiple AZs, graceful degradation

5. **Cloud is a trade-off, not a panacea**
   Evaluate fit for each workload

References & Further Reading

Essential Reading:

- Kleppmann – Designing Data-Intensive Applications (Ch 5, 7, 8, 9)
- Brewer – "CAP Twelve Years Later" (IEEE Computer, 2012)
- Vogels – "Eventually Consistent" (ACM, 2009)

Online Resources:

- Raft Visualization: raft.github.io
- Jepsen (distributed systems testing): jepsen.io
- CAP FAQ: the-paper-trail.org/page/cap-faq

Next Lecture: Cloud Service Models and Economics

Questions?

Exercise: Distributed Systems Foundations

~2 hours

CAP analysis • Consistency scenarios • Quorum calculations